

twitch.tv/dancojocar

youtube.com/dancojocar

Lecture #3

Navigation and Rest Resources

Mobile Applications
Fall 2024

REST using Retrofit

Retrofit

A type-safe HTTP client for Android and Java

Introduction

Retrofit turns your HTTP API into a Java interface.

```
public interface GitHubService {  
    @GET("users/{user}/repos")  
    Call<List<Repo>> listRepos(@Path("user") String user);  
}
```

The `Retrofit` class generates an implementation of the `GitHubService` interface.

```
Retrofit retrofit = new Retrofit.Builder()  
    .baseUrl("https://api.github.com/")  
    .build();  
  
GitHubService service = retrofit.create(GitHubService.class);
```

Each `Call` from the created `GitHubService` can make a synchronous or asynchronous HTTP request to the remote webserver.

```
Call<List<Repo>> repos = service.listRepos("octocat");
```

Use annotations to describe the HTTP request:

- URL parameter replacement and query parameter support
- Object conversion to request body (e.g., JSON, protocol buffers)
- Multipart request body and file upload

<https://square.github.io/retrofit/>

REST using Retrofit

```
implementation "com.squareup.retrofit2:retrofit:version"  
implementation "com.squareup.retrofit2:adapter-rxjava2:version"  
implementation "com.squareup.retrofit2:converter-gson:version"  
  
implementation "io.reactivex.rxjava2:rxandroid:version"
```

REST using Retrofit

DEMO

```
interface MovieService {
```

```
    @GET("movies")
```

```
    val movies: Observable<List<Movie>>
```

```
    @GET("genres")
```

```
    val genres: Observable<List<String>>
```

```
    @GET("moviesByGenre/{genre}")
```

```
    fun moviesByGenre(@Path("genre") genre: String)
        : Observable<List<Movie>>
```

```
    @GET("details/{id}")
```

```
    fun details(@Path("id") id: Int): Observable<Movie>
```

```
    @POST("updateDescription")
```

```
    fun updateDescription(@Body movie: Movie): Observable<Movie>
```

```
    @POST("updateRating")
```

```
    fun updateRating(@Body movie: Movie): Observable<Movie>
```

```
    @POST("update")
```

```
    fun update(@Body movie: Movie): Observable<Movie>
```

```
    @DELETE("delete/{id}")
```

```
    fun delete(@Path("id") id: Int): Observable<ResponseBody>
```

```
    @POST("add")
```

```
    fun add(@Body movie: Movie): Observable<Movie>
```

```
    companion object {
```

```
        const val SERVICE_ENDPOINT = "http://SERVER_IP:2022"
```

```
    }
```

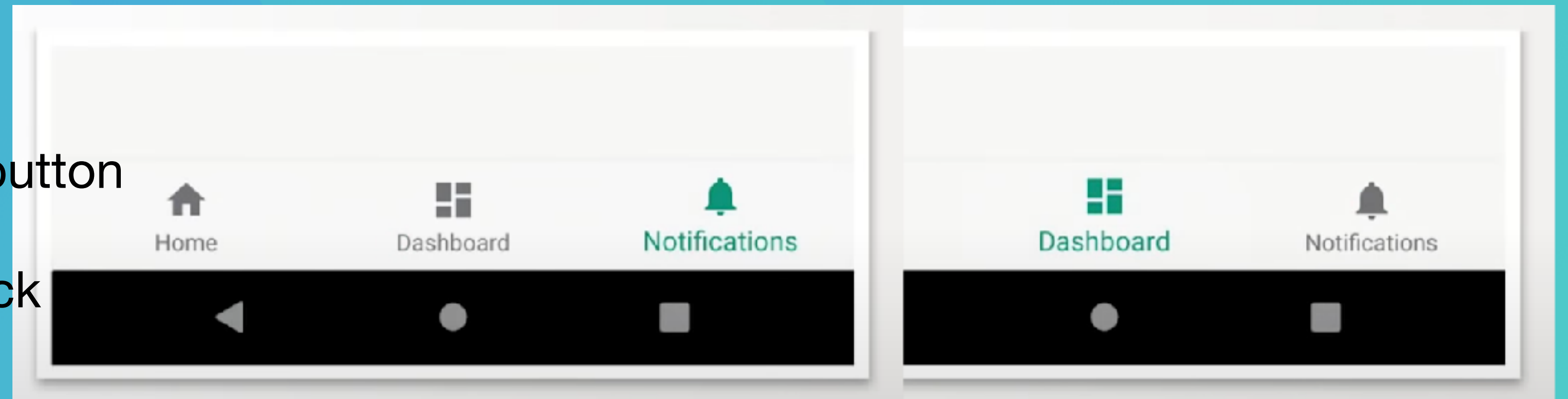
```
}
```

Navigation



Navigation

- Correctly
- Highlight the correct button
- Handles the back-stack



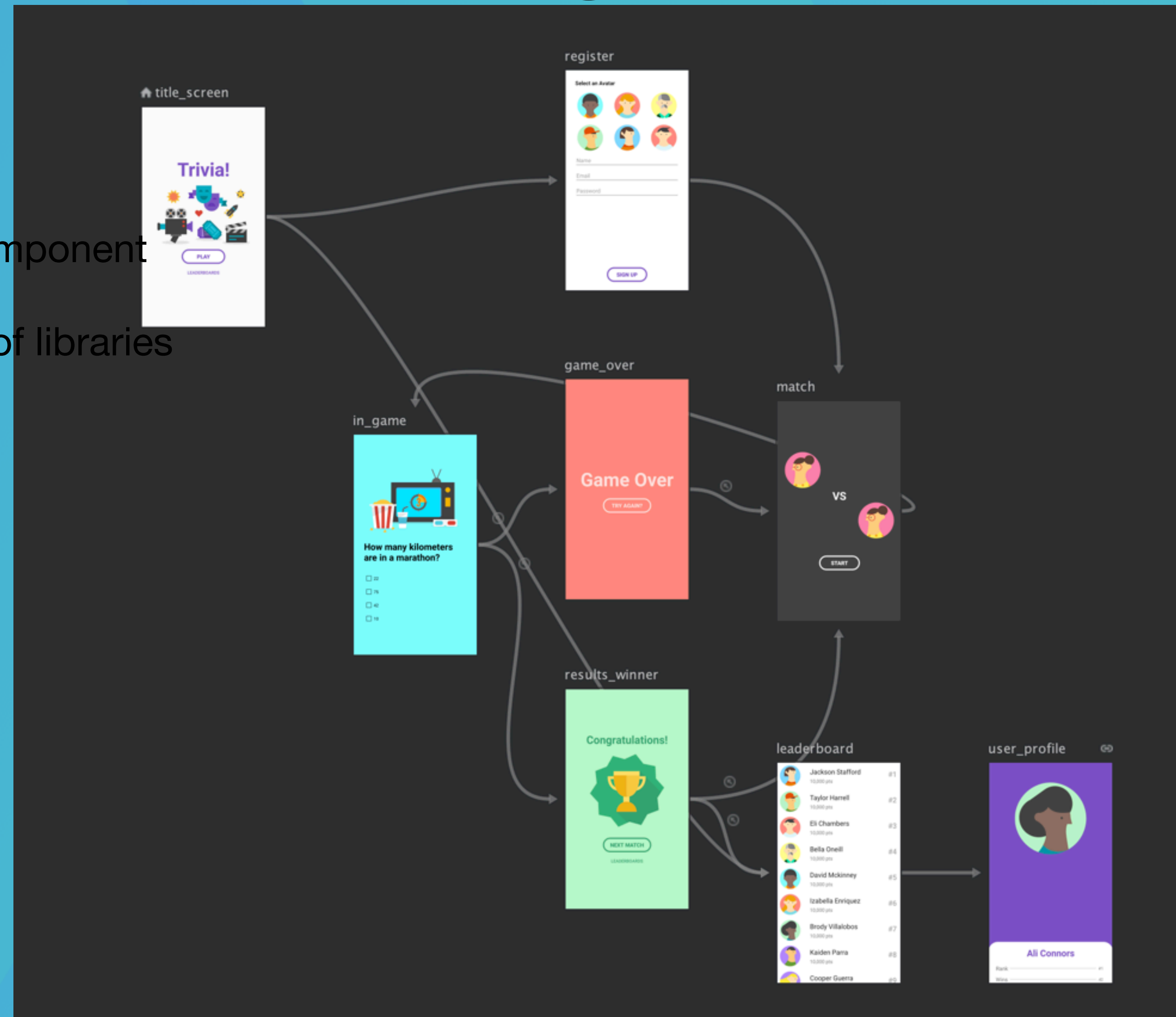
Key concepts

Concept	Purpose	Type
Host	A UI element that contains the current navigation destination. That is, when a user navigates through an app, the app essentially swaps destinations in and out of the navigation host.	• Compose: NavHost • Fragments: NavHostFragment
Graph	A data structure that defines all the navigation destinations within the app and how they connect together.	NavGraph
Controller	The central coordinator for managing navigation between destinations. The controller offers methods for navigating between destinations, handling deep links, managing the back stack, and more.	NavController
Destination	A node in the navigation graph. When the user navigates to this node, the host displays its content.	NavDestination Typically created when constructing the navigation graph.
Route	Uniquely identifies a destination and any data required by it. You can navigate using routes. Routes take you to destinations.	Any serializable data type.

Navigation

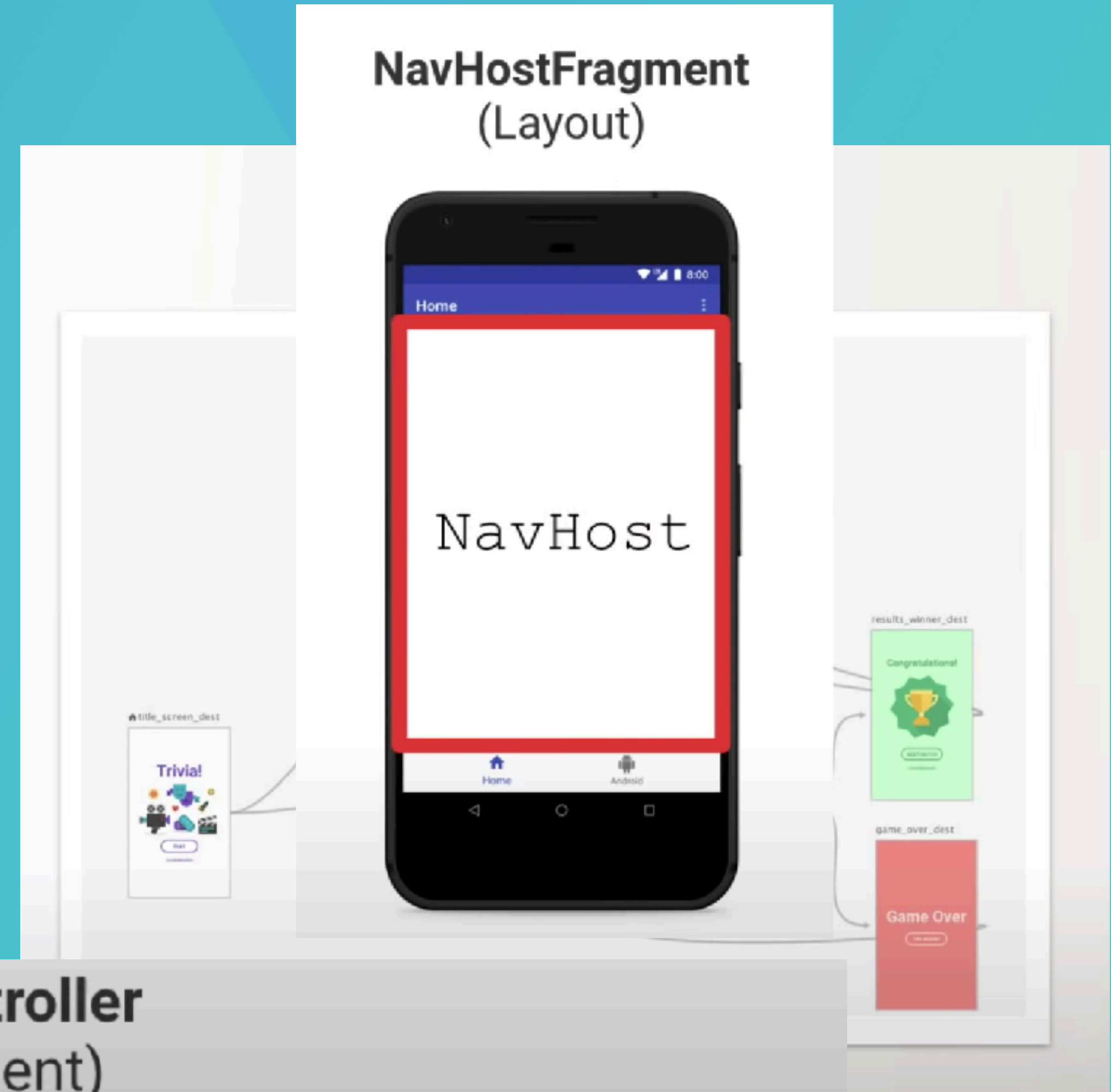
Navigation Component

- A collection of libraries
- A plugin
- Tooling



Navigation

- Navigation Graph
- NavHostFragment
- NavController



NavController
(Fragment)

```
findNavController().navigate(<Destination or Action id>)
```

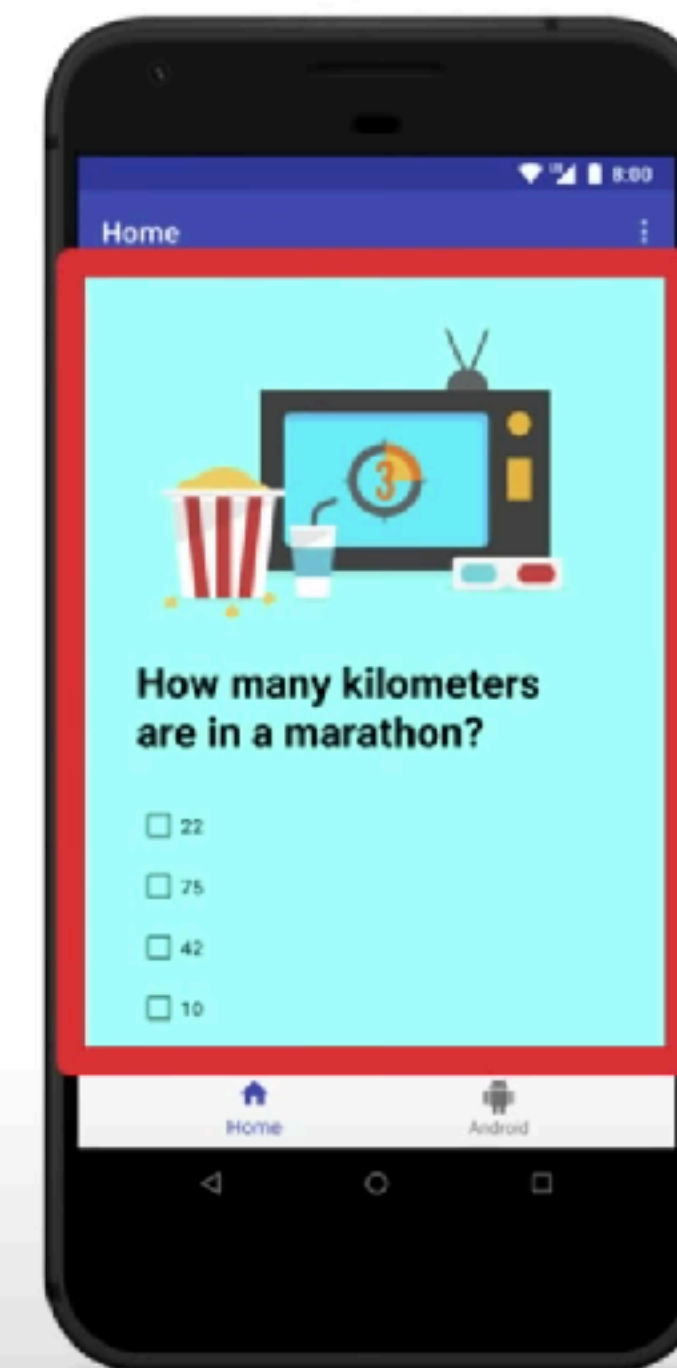
Navigation

DEMO

Navigation Graph
(New Resource)



NavHostFragment
(Layout)



NavController
(Fragment)

```
findNavController().navigate(R.id.win_action)
```

<https://developer.android.com/guide/navigation>



Navigation with Compose

```
dependencies {  
    val nav_version = "2.8.2"  
  
    implementation("androidx.navigation:navigation-compose:$nav_version")  
}
```

Navigate to a composable

```
@Serializable  
object FriendsList
```

```
navController.navigate(route = FriendsList)
```

```
@Serializable
object Profile
@Serializable
object FriendsList
@Composable
fun MyAppNavHost(
    modifier: Modifier = Modifier,
    navController: NavController = rememberNavController(),
) {
    NavHost(
        modifier = modifier,
        navController = navController,
        startDestination = Profile
    ) {
        composable<Profile> {
            ProfileScreen(
                onNavigateToFriends = { navController.navigate(route = FriendsList) },
                /* ... */
            )
        }
        composable<FriendsList> { FriendsListScreen(/* ... */) }
    }
}
@Composable
fun ProfileScreen(
    onNavigateToFriends: () -> Unit,
    /* ... */
) {
```

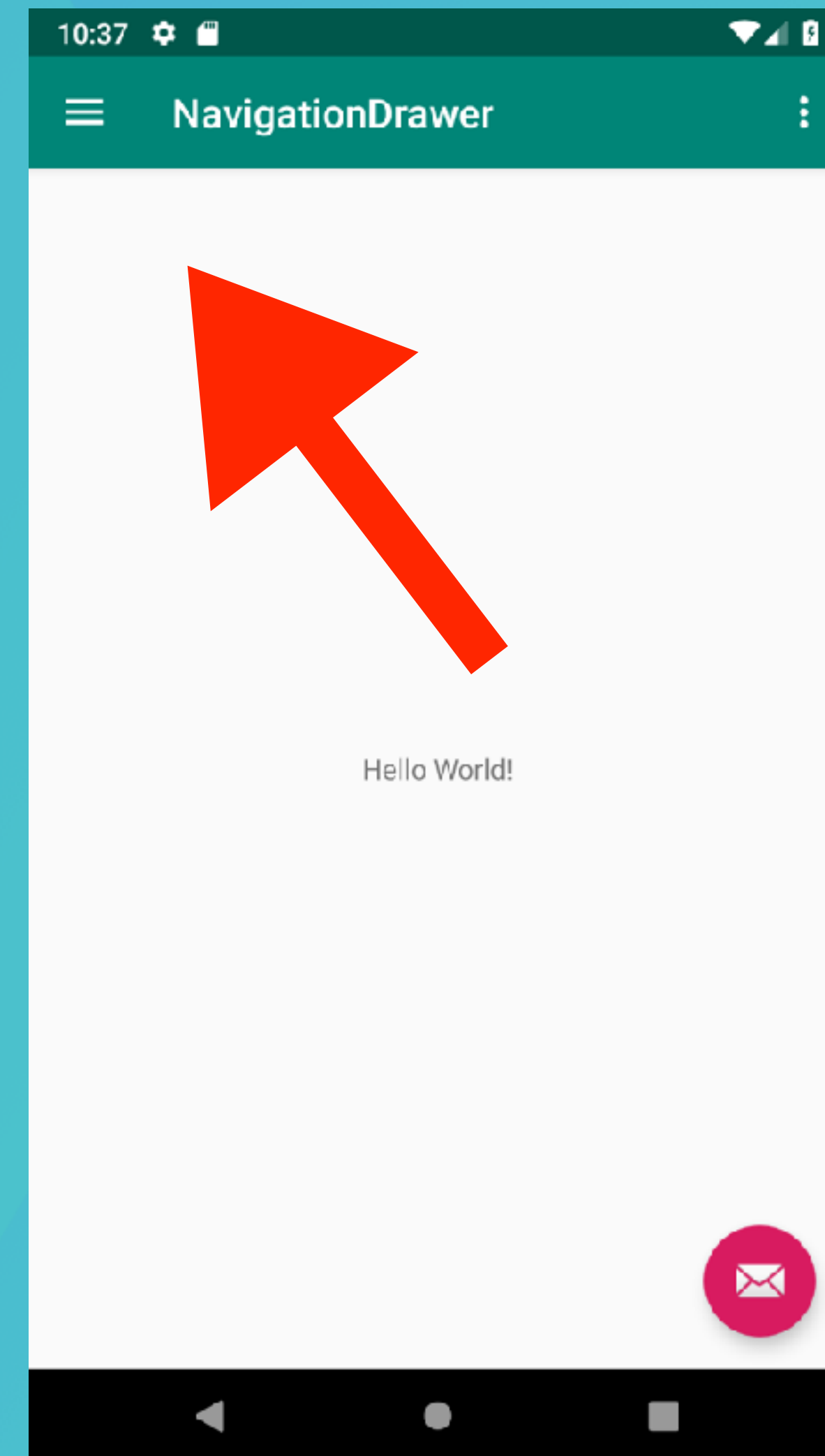
```
fun MyAppNavHost(
    modifier: Modifier = Modifier,
    navController: NavHostController = rememberNavController(),
) {
    NavHost(
        modifier = modifier,
        navController = navController,
        startDestination = Profile
    ) {
        composable<Profile> {
            ProfileScreen(
                onNavigateToFriends = { navController.navigate(route = FriendsList) },
                /* ... */
            )
        }
        composable<FriendsList> { FriendsListScreen(/* ... */) }
    }
}

@Composable
fun ProfileScreen(
    onNavigateToFriends: () -> Unit,
    /* ... */
) {
    /* ... */
    Button(onClick = onNavigateToFriends) {
        Text(text = "See friends list")
    }
}
```



Navigation Drawer

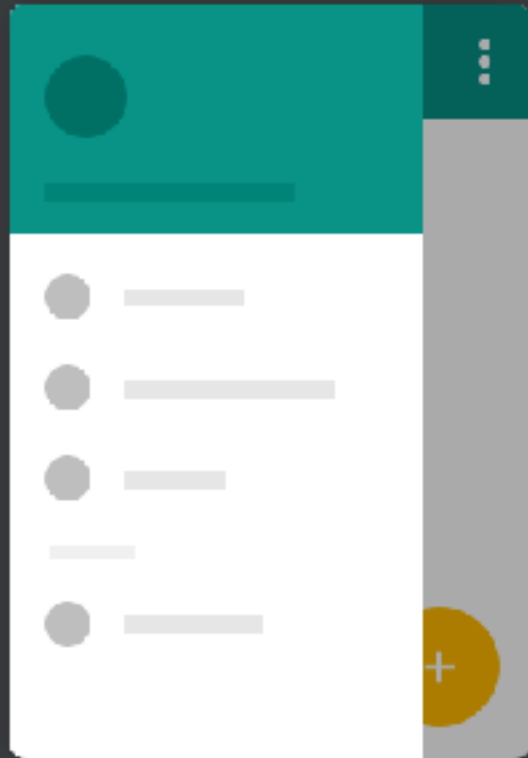
- App main navigation menu.
- Hidden when not in use.
- Appears:
 - with a left swipe from the screen edge
 - when the user touches the drawer icon in the app bar



Create New Project



Configure Your Project



Navigation Drawer Activity

Creates a new Activity with a Navigation Drawer

Name

Package name

Save location

Language

Kotlin

Minimum SDK

API 29: Android 10.0 (Q)

 Your app will run on approximately 8.2% of devices.
[Help me choose](#)

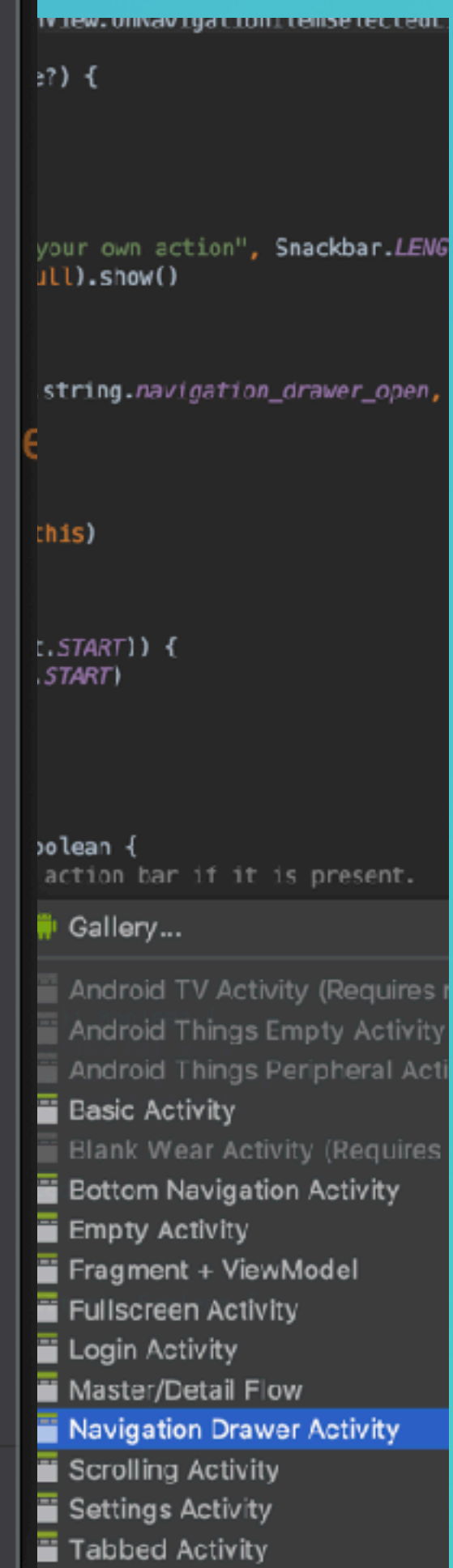
☐ Use legacy android.support libraries 

Cancel

Previous

Next

Finish





Android Studio
android.studio@android.com



Home



Gallery



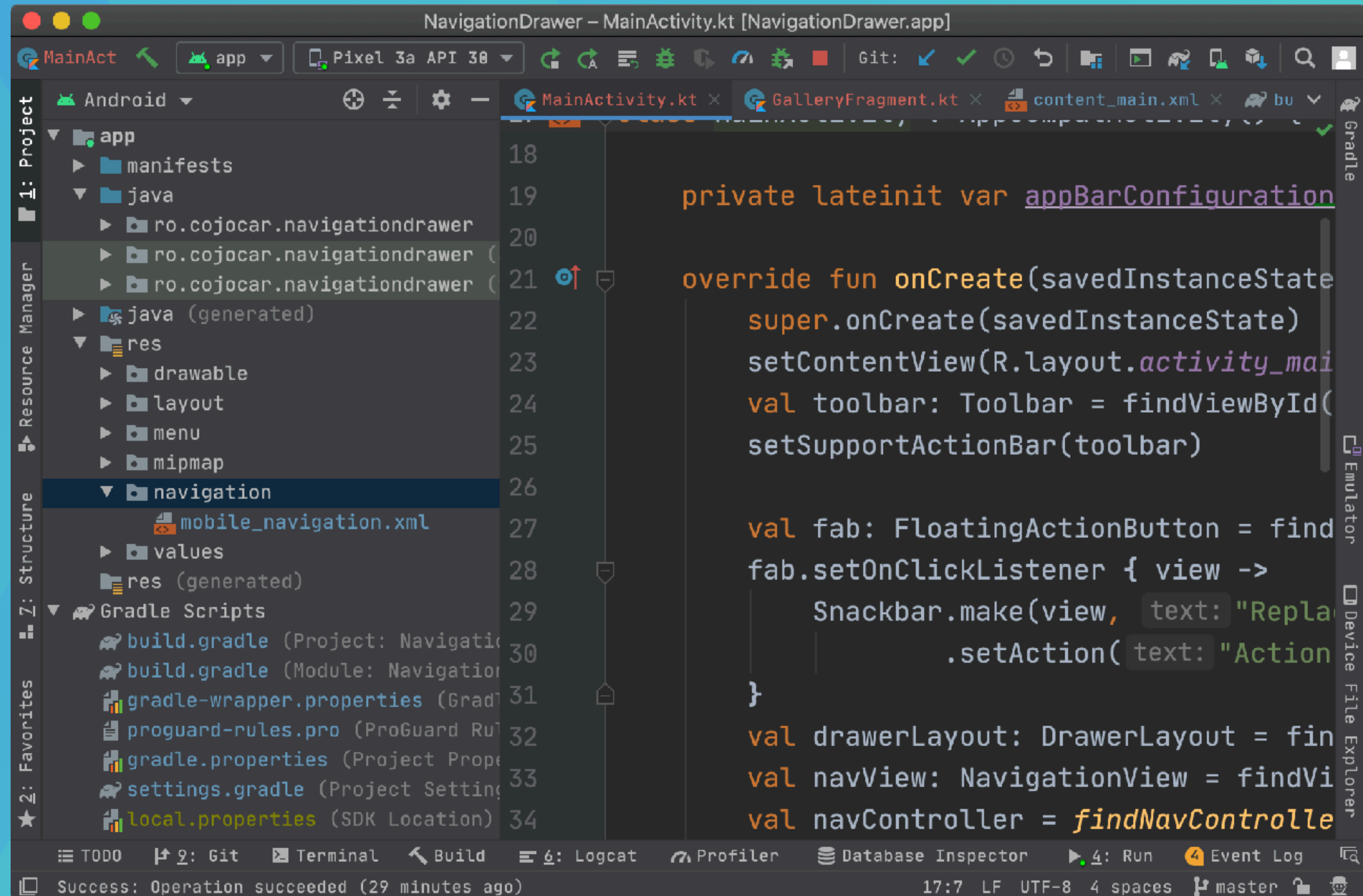
Slideshow

nt



Generated Artifacts

- Sources
- Layouts
- Menus
- Navigation



Dependencies

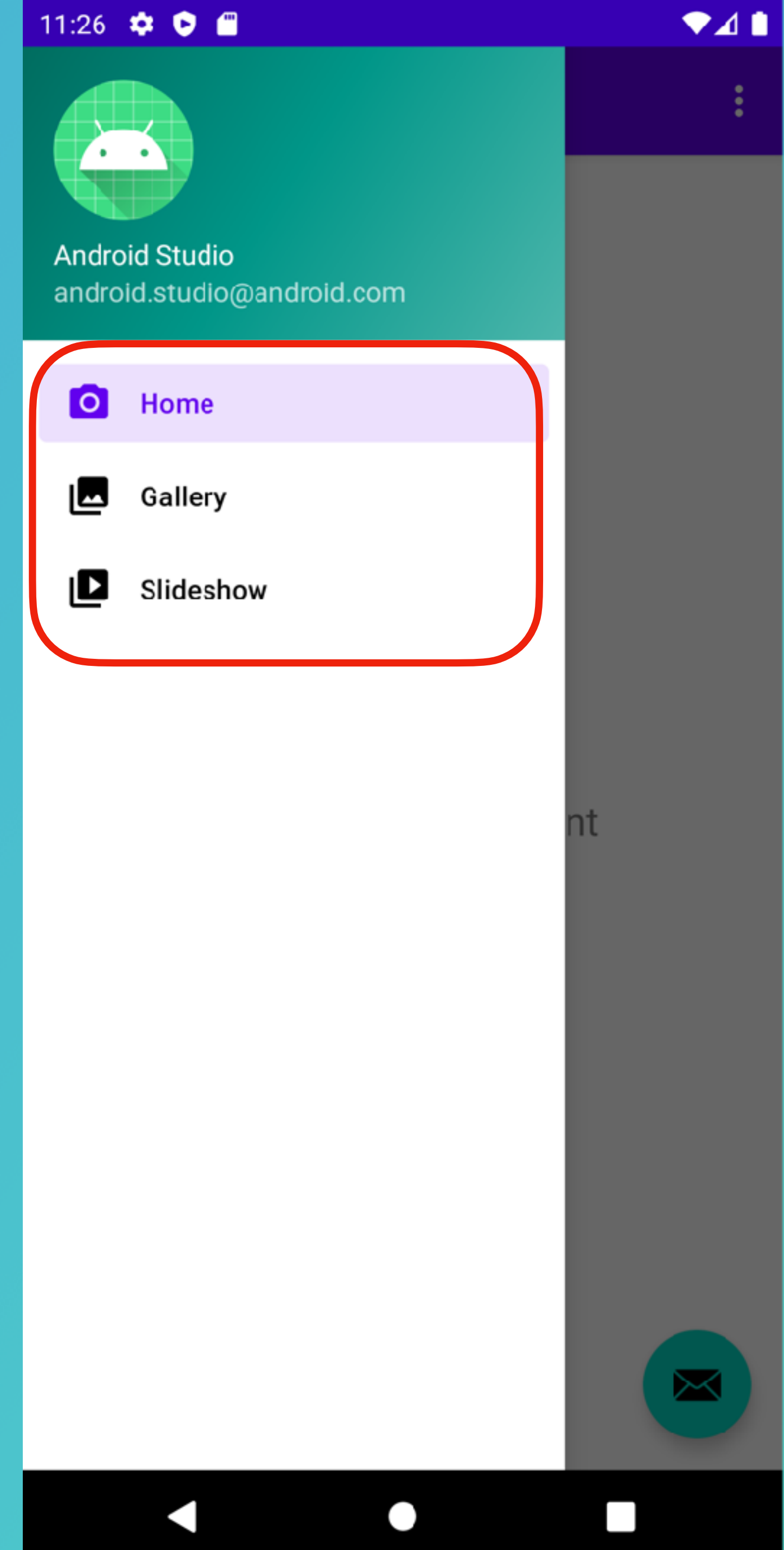
```
buildscript {  
    ext.nav_version = "2.5.3"  
}  
...  
dependencies {  
    implementation "androidx.navigation:navigation-fragment-ktx:$nav_version"  
    implementation "androidx.navigation:navigation-ui-ktx:$nav_version"  
}
```

Add a drawer to a layout

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.drawerlayout.widget.DrawerLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/drawer_layout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true"
    tools:openDrawer="start">
    <include
        layout="@layout/app_bar_main"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
    <com.google.android.material.navigation.NavigationView
        android:id="@+id/nav_view"
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:layout_gravity="start"
        android:fitsSystemWindows="true"
        app:headerLayout="@layout/nav_header_main"
        app:menu="@menu/activity_main_drawer" />
</androidx.drawerlayout.widget.DrawerLayout>
```

Declare the menu items

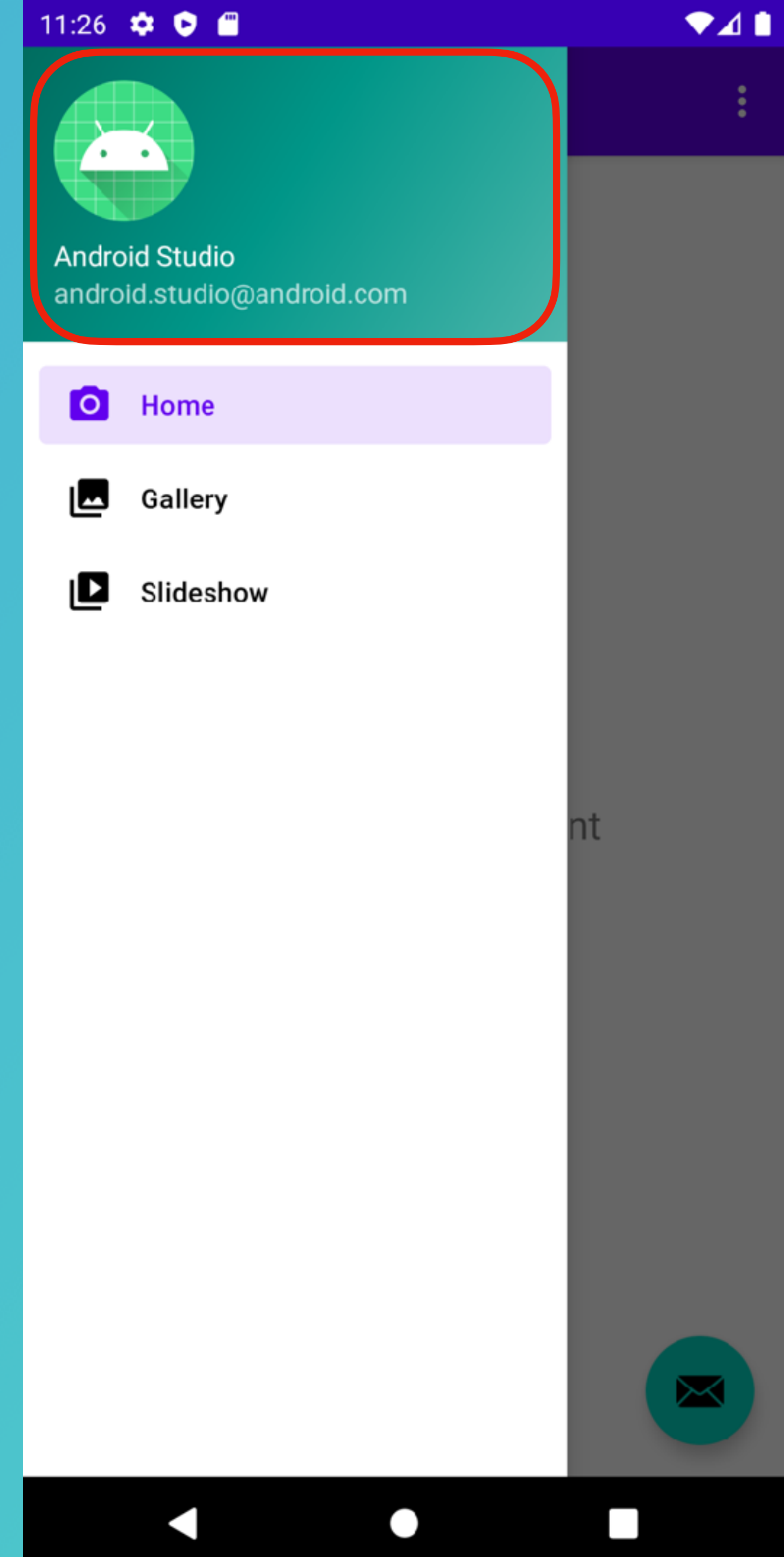
```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto"
      android:clickableBehavior="single">
    <item android:layout_width="wrap_content"
          android:layout_height="match_parent"
          android:layout_gravity="start"
          android:title="@string/nav_camera"
          android:fitsSystemWindows="true" />
    <item app:menu="@menu/activity_main_drawer" />
    <item android:id="@+id/nav_gallery"
          android:icon="@drawable/ic_menu_gallery"
          android:title="@string/gallery" />
    <item
        android:id="@+id/nav_slideshow"
        android:icon="@drawable/ic_menu_slideshow"
        android:title="@string/slideshow" />
    ...
</group>
</menu>
```



Add a header to the nav drawer

```
<android.support.design.widget.NavigationView
    android:id="@+id/nav_view"
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:layout_gravity="start"
    android:fitsSystemWindows="true"
    app:headerLayout="@layout/nav_header_main" />
    android:background="?attr/colorPrimaryDark"
    android:padding="16dp"
    android:theme="@style/ThemeOverlay.AppCompat.Dark"
    android:orientation="vertical"
    android:gravity="bottom">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="My header title"
        android:textAppearance=
            "@style/TextAppearance.AppCompat.Body1" />
</LinearLayout>
```



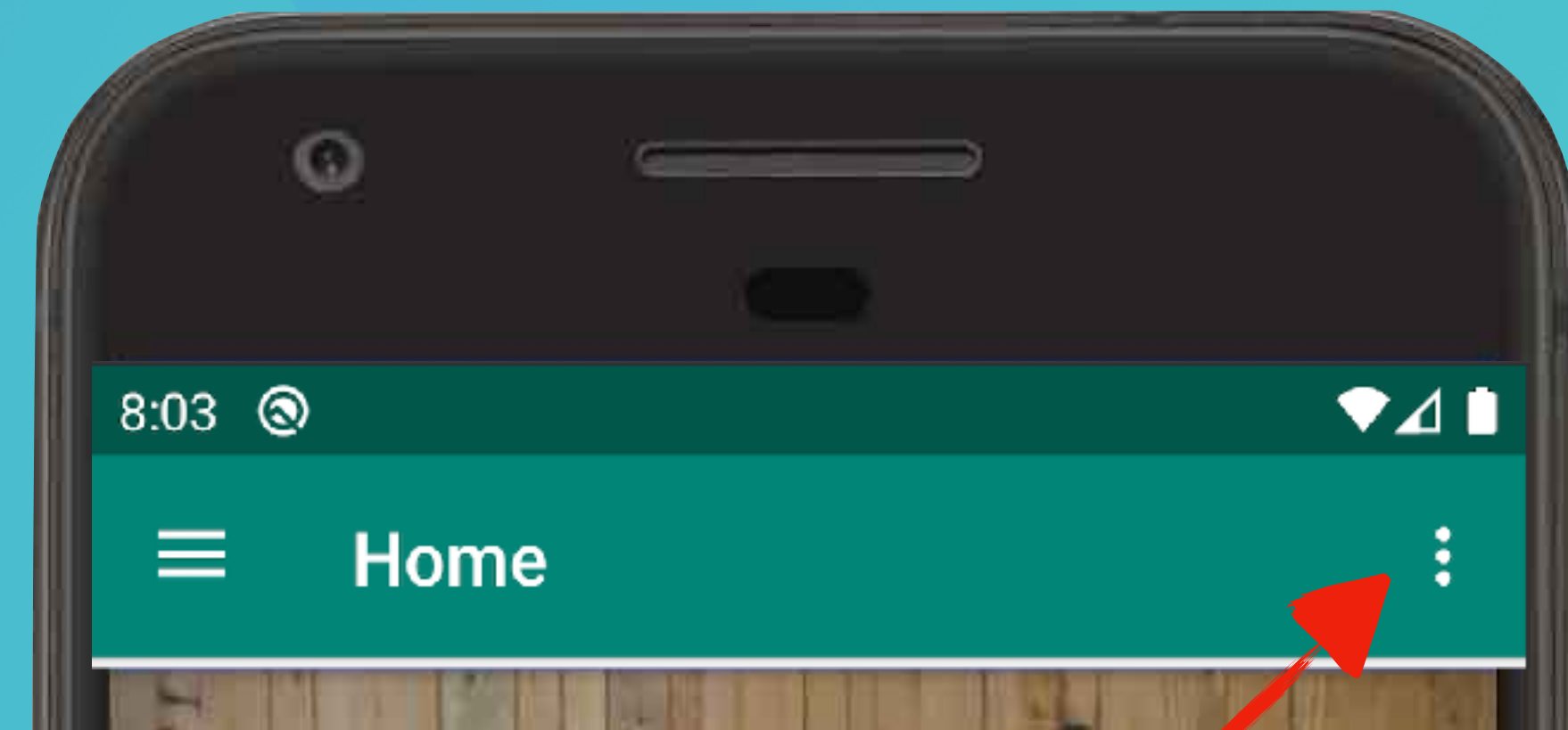
Handle navigation events

```
class MainActivity : AppCompatActivity() {  
  
    private lateinit var appBarConfiguration: AppBarConfiguration  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
        setSupportActionBar(toolbar)  
  
        val navController = findNavController(R.id.nav_host_fragment)  
  
        appBarConfiguration = AppBarConfiguration(  
            listOf(  
                R.id.nav_home,  
                R.id.nav_gallery  
            ), drawer_layout  
        )  
        setupActionBarWithNavController(navController, appBarConfiguration)  
        nav_view.setupWithNavController(navController)  
    }  
}
```

Add a toolbar



```
<?xml version="1.0" encoding="utf-8"?>
<class MainActivity : AppCompatActivity() {
<androidx.coordinatorlayout.widget.CoordinatorLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:supportActionBar="false"
tools:context=".MainActivity" >
    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:theme="@style/Theme.NoActionBar" >
        <include layout="@layout/content" />
        <android.support.design.widget.Toolbar
            android:id="@+id/navigation_drawer_close"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:background="@attr/colorPrimary"
            app:popupTheme="@style/AppTheme.PopupOverlay" />
    </com.google.android.material.appbar.AppBarLayout>
```



```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        ...  
        setSupportActionBar(toolbar)  
        ...  
    }  
}
```

Other State Changes

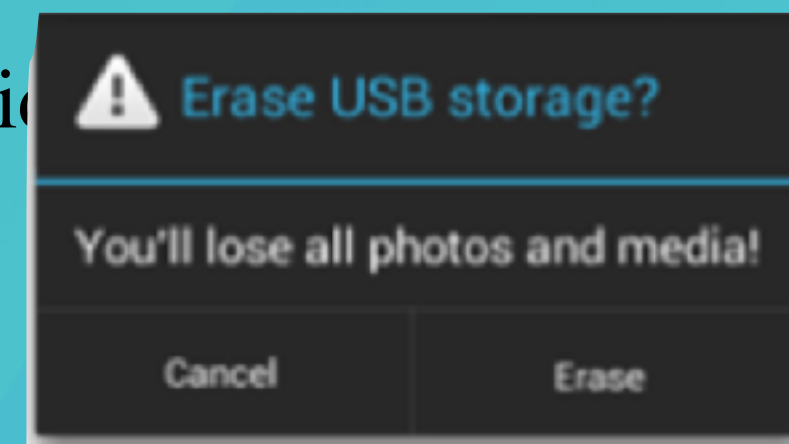
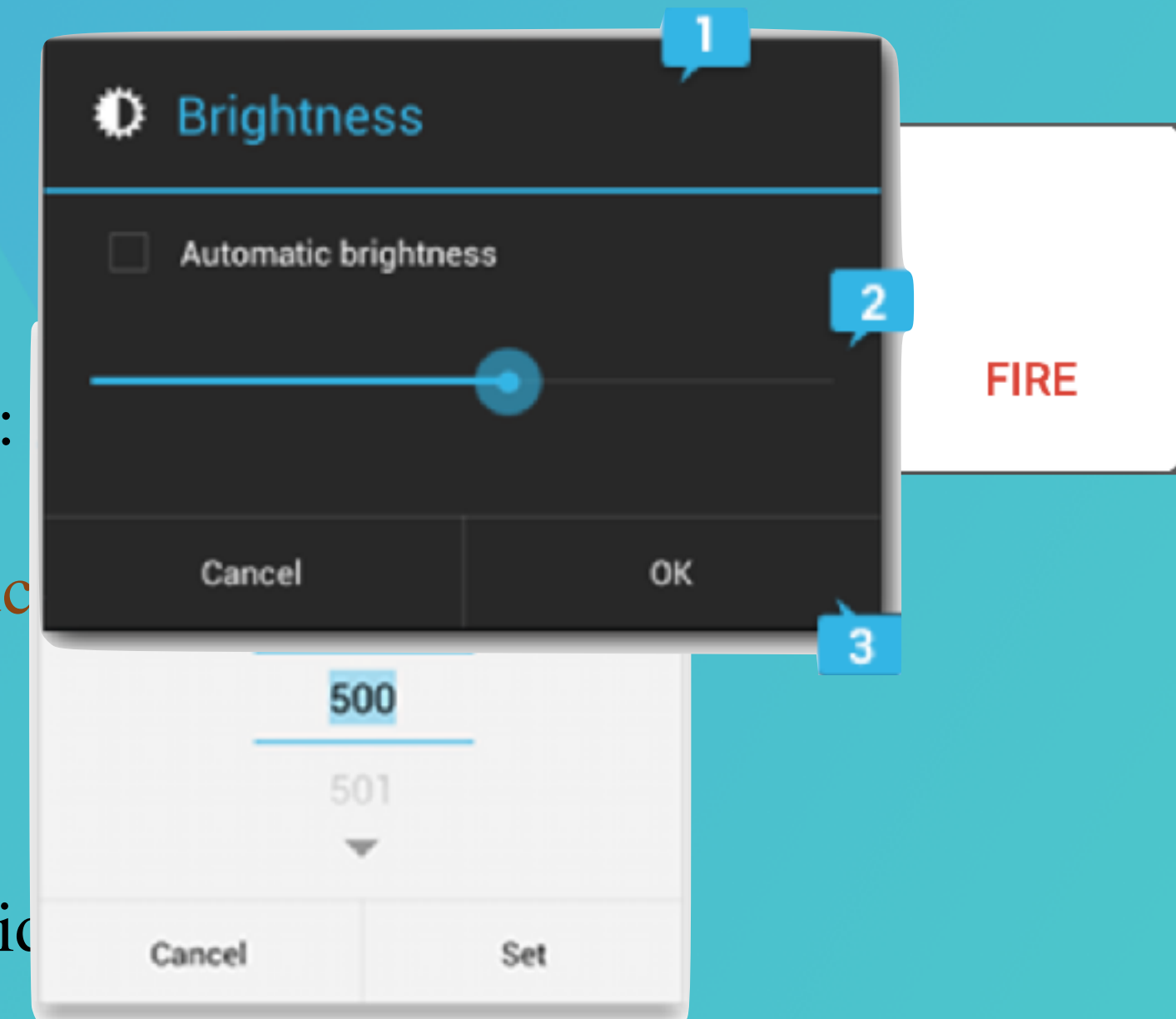


```
drawer_layout.addDrawerListener(  
    object : DrawerLayout.DrawerListener {  
        override fun onDrawerSlide(drawerView: View, slideOffset: Float) {  
            // Respond when the drawer's position changes  
        }  
  
        override fun onDrawerOpened(drawerView: View) {  
            // Respond when the drawer is opened  
        }  
  
        override fun onDrawerClosed(drawerView: View) {  
            // Respond when the drawer is closed  
        }  
  
        override fun onDrawerStateChanged(newState: Int) {  
            // Respond when the drawer motion state changes  
        }  
    }  
)
```

Dialogs

```
class FireMissilesDialogFragment : DialogFragment() {

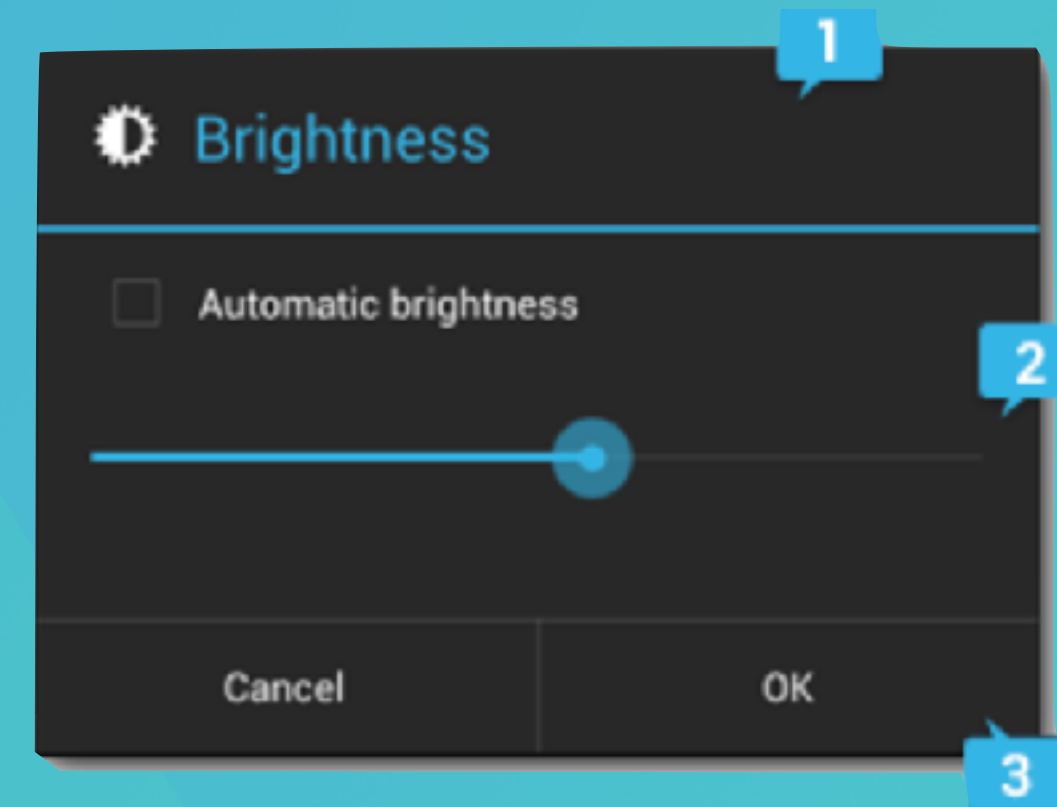
    override fun onCreateDialog(savedInstanceState: Bundle):
        return activity?.let {
            // Use the Builder class for convenient dialog construction
            // 1. Instantiate a Dialog Builder in its constructor
            val builder = AlertDialog.Builder(activity).for {missiles)
            builder.setTitle(R.string.dialog_title)
            builder.setPositiveButton(R.string.fire,
            { dialog, id, _ -> // FIRE ZE MISSILES!
            // 2. Chain together various setter methods to set the dialog characteristics
            builder.setMessage(R.string.dialog_message)
            builder.setNegativeButton(R.string.cancel,
            { dialog, id, _ -> // User cancelled the dialog
            // 3. Get the AlertDialog from create()
            val dialog = AlertDialog.Builder(activity).create()
            builder.create()
            } ?: throw IllegalStateException("Activity cannot be null")
            }
        }
```



<https://developer.android.com/guide/topics/ui/dialogs>

Adding actions

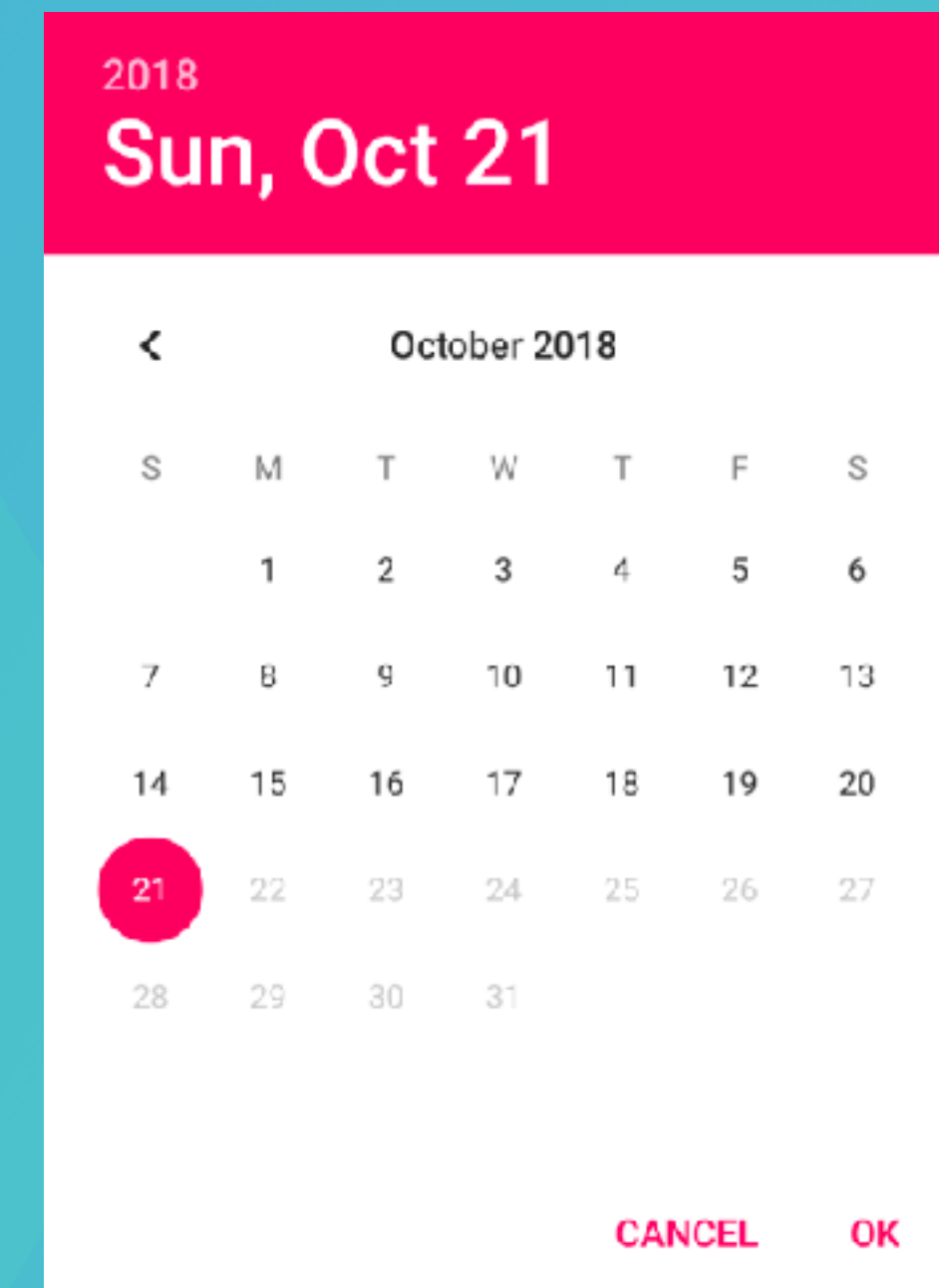
```
val alertDialog: AlertDialog? = activity?.let {  
    val builder = AlertDialog.Builder(it)  
    builder.apply {  
        setPositiveButton(R.string.ok,  
            DialogInterface.OnClickListener { dialog, id ->  
                // User clicked OK button  
            })  
        setNegativeButton(R.string.cancel,  
            DialogInterface.OnClickListener { dialog, id ->  
                // User cancelled the dialog  
            })  
    }  
    // Set other dialog properties  
    ...  
  
    // Create the AlertDialog  
    builder.create()  
}
```



<https://developer.android.com/guide/topics/ui/dialogs>

Using Anko

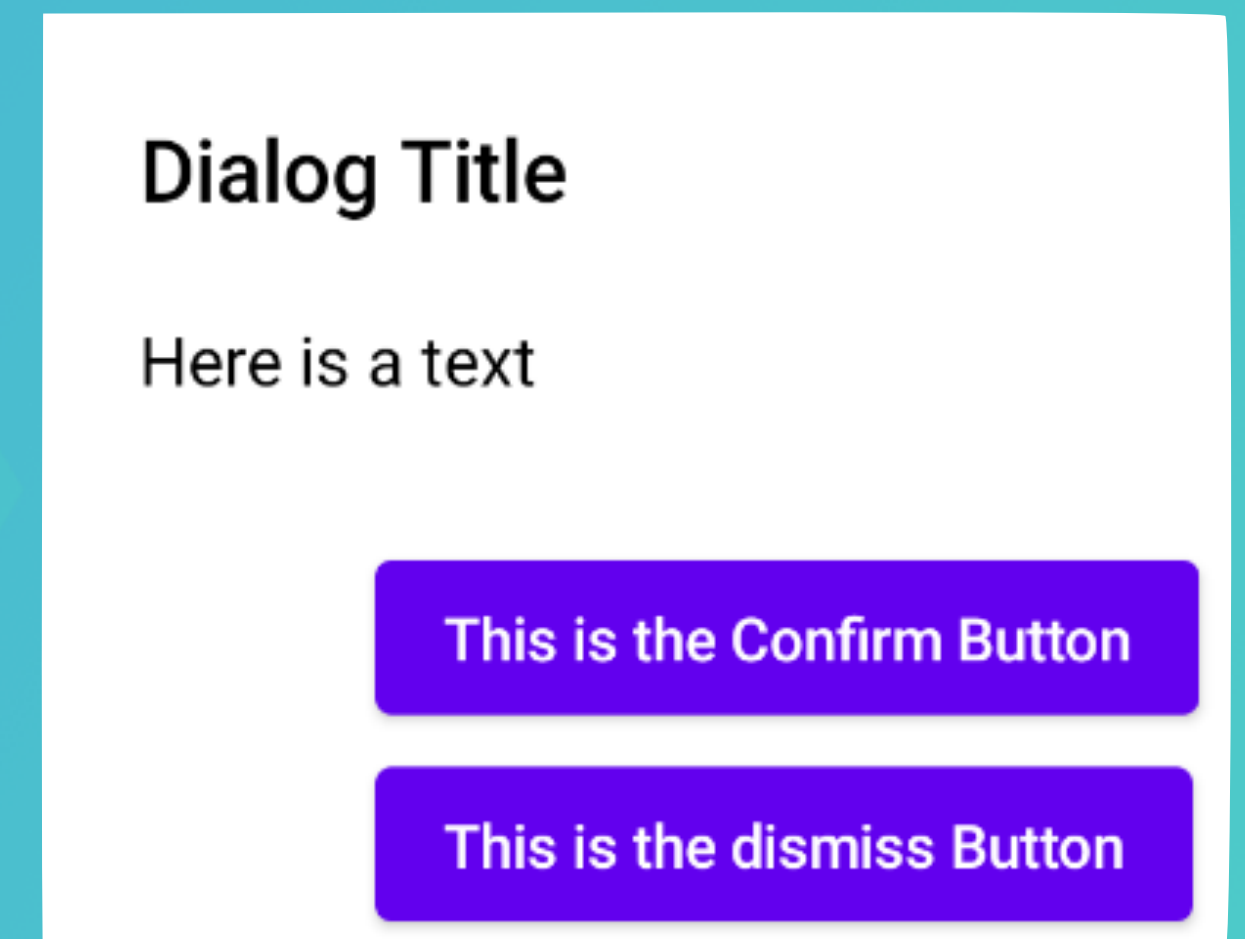
```
alert {
    isCancelable = false
    lateinit var datePicker: DatePicker
    customView {
        verticalLayout {
            datePicker = datePicker {
                maxDate = System.currentTimeMillis()
            }
        }
    }
    yesButton {
        val parsedDate =
            "${datePicker.dayOfMonth}/${datePicker.month + 1}/${datePicker.year}"
        toast("Selected date: $parsedDate")
    }
    noButton { }
}.show()
```



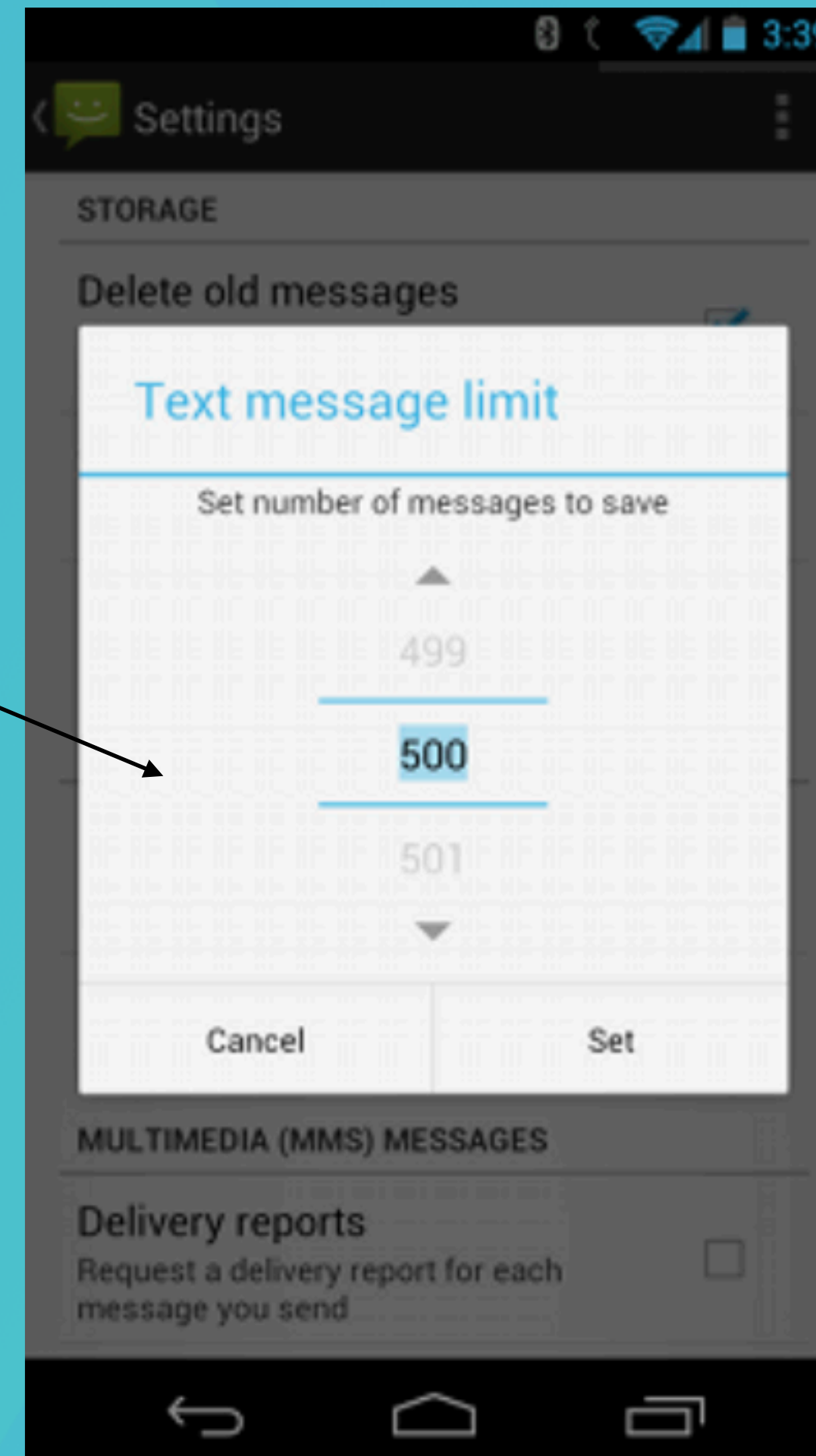
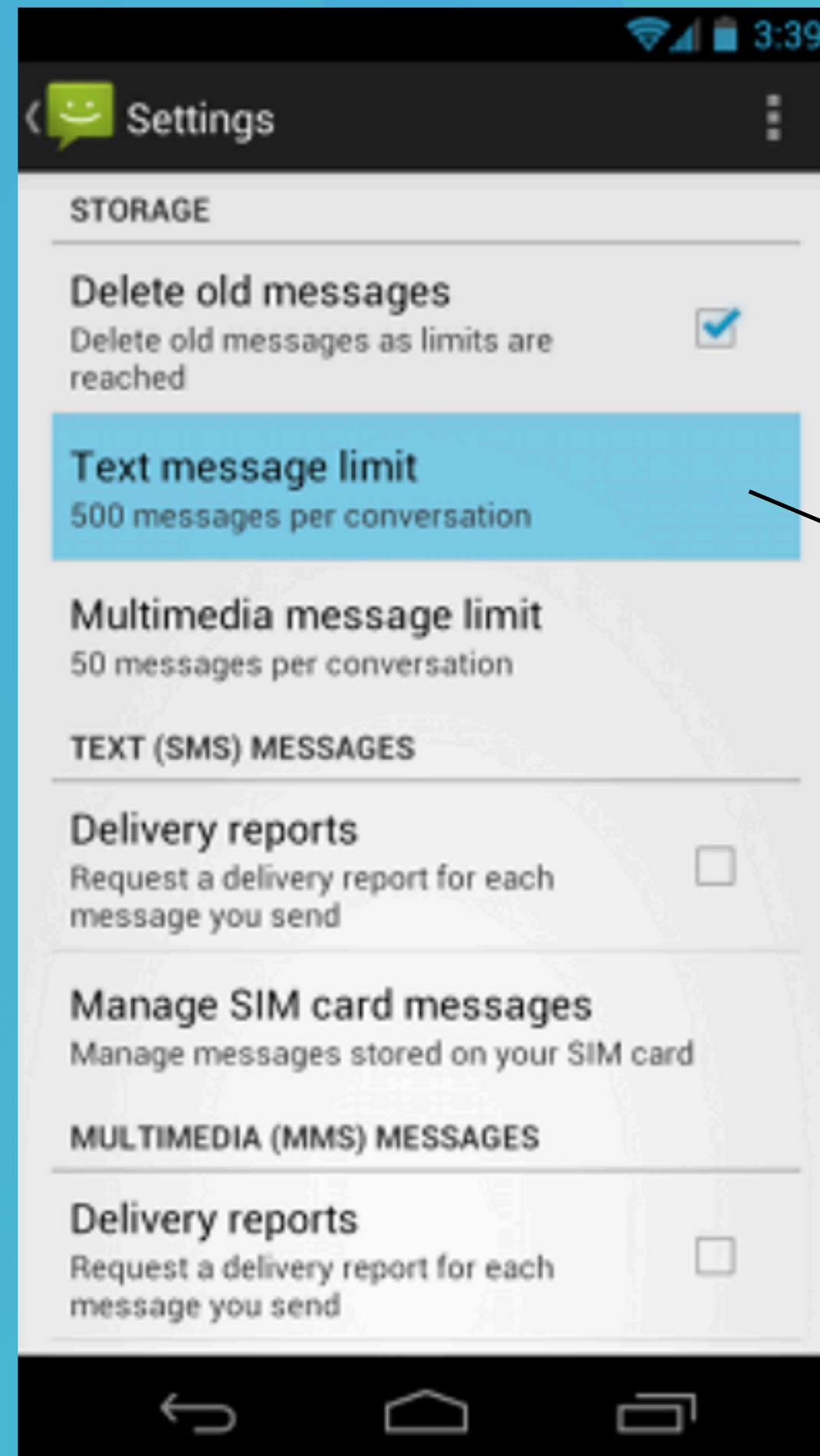
<https://github.com/Kotlin/anko/wiki/Anko-Commons---Dialogs>

Jetpack Compose

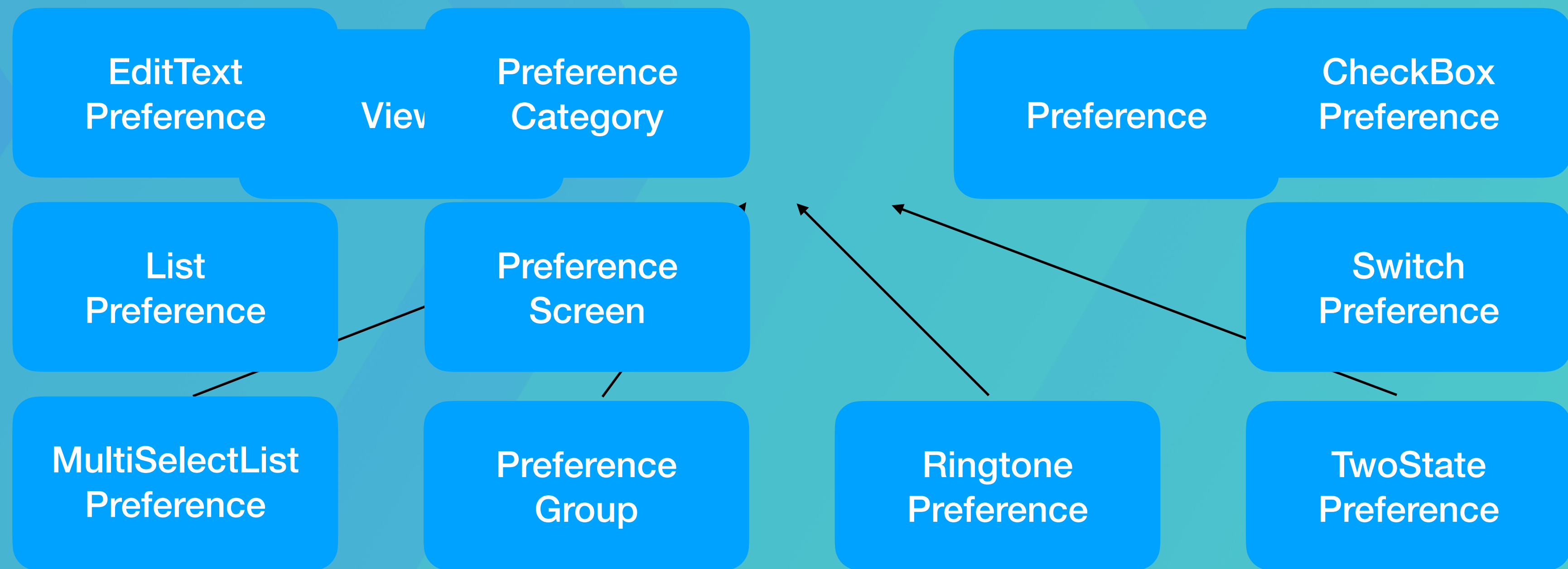
```
AlertDialog(  
    onDismissRequest = {  
        // Dismiss the dialog when the user clicks outside the dialog or on the back  
        // button. If you want to disable that functionality, simply use an empty  
        // onCloseRequest.  
        openDialog.value = false  
    },  
    title = { Text(text = "Dialog Title") },  
    text = { Text("Here is a text ") },  
    confirmButton = {  
        Button(  
            onClick = { openDialog.value = false }  
        ) { Text("This is the Confirm Button") }  
    },  
    dismissButton = {  
        Button(  
            onClick = { openDialog.value = false }  
        ) { Text("This is the dismiss Button") }  
    }  
)
```



Preferences



Preferences



<https://developer.android.com/reference/android/preference/Preference>

Preferences

Preference

String

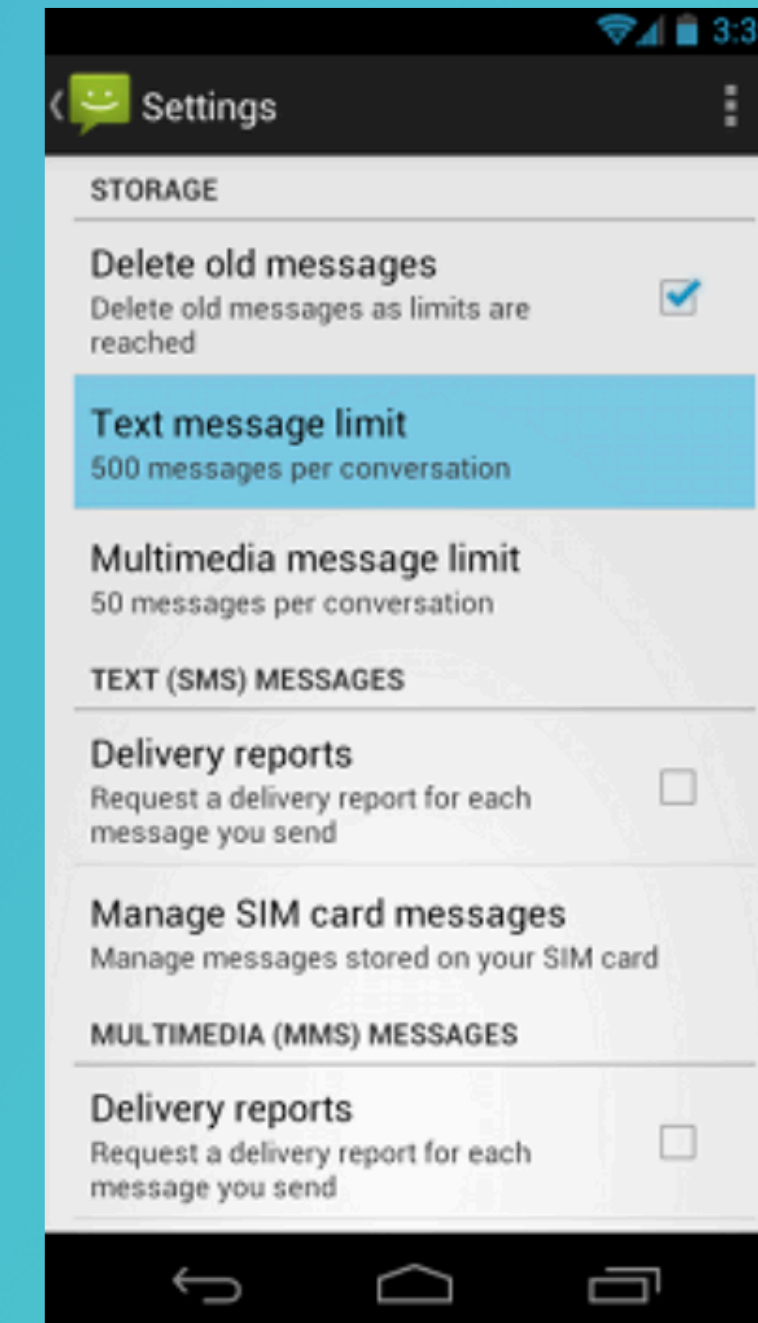
Set<String>

DEMO

PreferenceScreen

```
<?xml version="1.0" encoding="utf-8"?>
<PreferenceScreen
    xmlns:android="http://schemas.android.com/apk/res/android">
    <CheckBoxPreference
        android:key="pref_sync"
        android:title="@string/pref_sync"
        android:summary="@string/pref_sync_summ"
        android:defaultValue="true" />
    <ListPreference
        android:dependency="pref_sync"
        android:key="pref_syncConnectionType"
        android:title="@string/pref_syncConnectionType"
        android:dialogTitle="@string/pref_syncConnectionType"
        android:entries="@array/pref_syncConnectionTypes_entries"
        android:entryValues="@array/pref_syncConnectionTypes_values"
        android:defaultValue="@string/pref_syncConnectionTypes_default" />
</PreferenceScreen>
</PreferenceScreen>

class SettingsActivity : PreferenceActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        addPreferencesFromResource(R.xml.preferences)
    }
}
```



Jetpack DataStore

Feature	SharedPreferences	Preferences DataStore	Proto DataStore
Async API	✓ (only for reading changed values, via listener)	✓ (via Flow)	✓ (via Flow)
Synchronous API	✓ (but not safe to call on UI thread)	✗	✗
Safe to call on UI thread	✗*	✓ (work is moved to Dispatchers.IO under the hood)	✓ (work is moved to Dispatchers.IO under the hood)
Can signal errors	✗	✓	✓
Safe from runtime exceptions	✗**	✓	✓
Has a transactional API with strong consistency guarantees	✗	✓	✓
Handles data migration	✗	✓ (from SharedPreferences)	✓ (from SharedPreferences)
Type safety	✗	✗	✓ with Protocol Buffers

Using DataStore

```
// Preferences DataStore  
implementation "androidx.datastore:datastore-preferences:1.0.0"
```

Create the DataStore

```
// with Preferences DataStore  
val datastore: DataStore<Preferences> = context.createDataStore(  
    name = "settings"  
)
```

Read Data

```
val MY_COUNTER = preferencesKey<Int>("my_counter")  
val myCounterFlow: Flow<Int> = datastore.data  
    .map { currentPreferences ->  
        currentPreferences[MY_COUNTER] ?: 0  
    }
```

Using DataStore

Write Data

```
suspend fun incrementCounter() {  
    datastore.edit { settings ->  
        // We can safely increment our counter without losing data due to races!  
        val currentCounterValue = settings[MY_COUNTER] ?: 0  
        settings[MY_COUNTER] = currentCounterValue + 1  
    }  
}
```

Using DataStore



Write Data

```
suspend fun incrementCounter() {  
    datastore.edit { settings ->  
        // We can safely increment our counter without losing data due to races!  
        val currentCounterValue = settings[MY_COUNTER] ?: 0  
        settings[MY_COUNTER] = currentCounterValue + 1  
    }  
}
```

Migrate from SharedPreferences

```
val datastore: DataStore<Preferences> = context.createDataStore(  
    name = "settings",  
    migrations = listOf(SharedPreferencesMigration(context, "settings_preferences"))  
)
```

Saving & Reading Local Files

- Internal storage
 - Internal cache files
- External storage
- Shared preferences
- Databases



<https://developer.android.com/guide/topics/data>

Internal Storage

- It's always available.
- Available only to your app.
- On uninstall everything is removed.



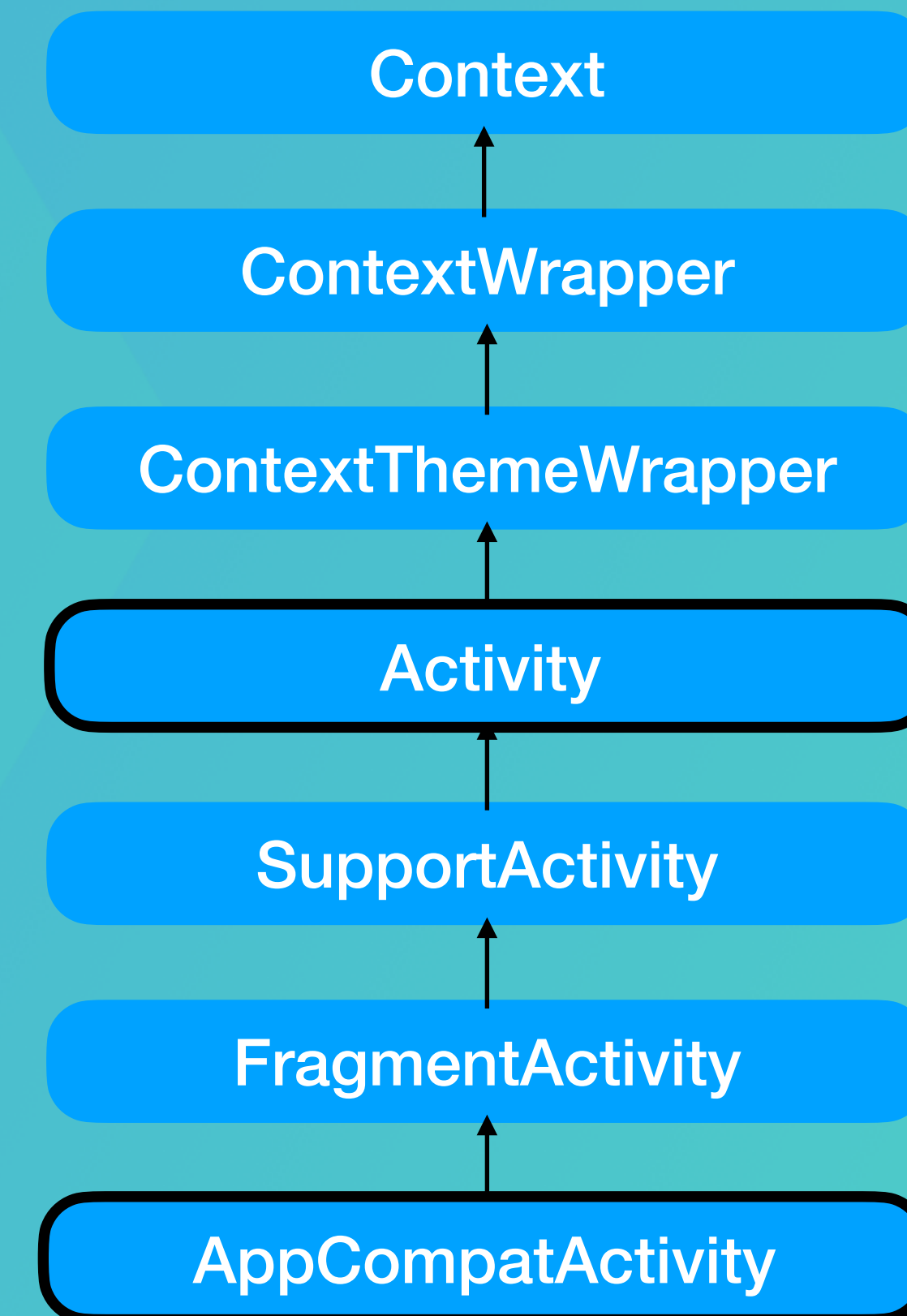
Neither the user nor other apps can access your files!

<https://developer.android.com/guide/topics/data/data-storage#filesInternal>

Internal Storage



```
public abstract class Context {  
    ...  
    public abstract File getFilesDir();  
    ...  
    public abstract File getCacheDir();  
}  
  
val file = File(context.filesDir, filename)  
val filename = "myfile"  
val fileContents = "Hello world!"  
context.openFileOutput(filename,  
    Context.MODE_PRIVATE).use {  
    it.write(fileContents.toByteArray())  
}  
  
private fun getTempFile(  
    context: Context, url: String): File? =  
    Uri.parse(url)?.lastPathSegment?.let { filename ->  
        File.createTempFile(filename, null, context.cacheDir)  
    }
```



External Storage Permissions

```
<manifest ...>
  <uses-permission android:name=
    "android.permission.WRITE_EXTERNAL_STORAGE" />
  <uses-permission android:name=
    "android.permission.READ_EXTERNAL_STORAGE" />
</manifest>
...
</manifest>
```

< Android 4.4 (API level 19)

```
/* Checks if external storage is available for read and write */
fun isExternalStorageWritable(): Boolean {
    return Environment.getExternalStorageState() == Environment.MEDIA_MOUNTED
}
/* Checks if external storage is available to at least read */
fun isExternalStorageReadable(): Boolean {
    return Environment.getExternalStorageState() in
        setOf(Environment.MEDIA_MOUNTED, Environment.MEDIA_MOUNTED_READ_ONLY)
}
```

<https://developer.android.com/training/data-storage/files#WriteInternalStorage>



FileProvider

content://com.example.myapp.fileprovider/myimages/default_image.jpg

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapp">
    <application
        ...>
        <provider
            android:name="android.support.v4.content.FileProvider"
            android:authorities="com.example.myapp.fileprovider"
            android:grantUriPermissions="true"
            android:exported="false">
            <meta-data
                android:name="android.support.FILE_PROVIDER_PATHS"
                android:resource="@xml/filepaths" />
            </provider>
            ...
        </application>
    </manifest>
    <paths>
        <files-path path="images/" name="myimages" />
    </paths>
```

<https://developer.android.com/training/secure-file-sharing/setup-sharing>

Lecture outcomes

- Navigate between screens/views.
- Use dialogs and pickers.
- Manage files & preferences.

